

PATRÓN DAO

¿Qué es un Patrón en desarrollo de software?

Un patrón es un **par problema/solución** con nombre que se puede aplicar en nuevos contextos, con consejos acerca de cómo aplicarlo en nuevas situaciones y discusiones sobre sus compromisos. Todos los patrones, idealmente, tienen nombres sugerentes.

Es decir que, un patrón es una solución general y reutilizable para un problema que aparece frecuentemente durante el desarrollo de software.

No es código que simplemente se copia y pega, sino una forma estructurada de organizar las clases y sus responsabilidades para resolver determinado problema.

¿Qué es el patrón de acceso a datos DAO?

DAO significa **Data Access Object** (Objeto de Acceso a Datos).

Es un patrón que tiene como objetivo separar la lógica de acceso a los datos del resto de la aplicación.

La idea fundamental es que las clases que representan la lógica de negocio no tengan que preocuparse por cómo se almacenan, consultan, modifican o eliminan los datos.

¿Para qué sirve?

El patrón DAO sirve principalmente para **centralizar y encapsular el acceso a los datos**.

Por ejemplo, un ClienteDAO podría tener métodos como:

```
crear(cliente)
buscarPorId(id)
listar()
actualizar(cliente)
eliminar(id)
```

La clase que utiliza el DAO no necesita saber si internamente esos métodos utilizan SQL, un archivo, una API u otro mecanismo de almacenamiento.

Esto aporta varias ventajas como podrían ser:

- **Separación de responsabilidades**, es decir, que cada clase se ocupe de una tarea específica y no haya una clase que mezcle tareas relacionadas a lógica de negocio con lógica de manejo de datos.
- **Favorece la reutilización**, ya que, diferentes partes del sistema pueden utilizar la misma clase DAO para acceder a los datos.
- **Existe un menor acoplamiento**, ya que, la lógica de negocio no depende directamente de una tecnología concreta de almacenamiento.

Realizar de forma gráfica su funcionamiento.

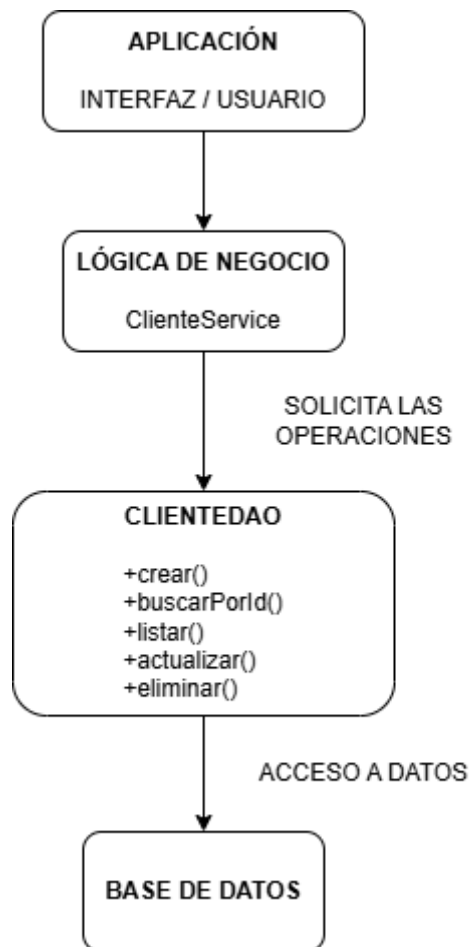


DIAGRAMA HECHO EN: <https://app.diagrams.net/>

ClienteService se ocupa de la lógica del sistema, mientras que ClienteDAO se ocupa específicamente de acceder a los datos de los clientes.

Hagamos un flujo de recorrido a modo de ejemplo y terminar de entender el concepto:

El **USUARIO** consulta en la **APLICACIÓN** el cliente con ID n° 23. **ClienteService** solicita la operación de búsqueda por ID que se encarga de realizar **CLIENTEDAO**

en la **BASE DE DATOS** a través de una query que ClienteService no sabe porque no es parte de sus responsabilidades saberlo. La Base de Datos devuelve los datos, CLIENTEDAO convierte esos datos en un objeto Cliente y se le vuelve a presentar, de forma amigable al usuario a través de la aplicación.